

Tp3: Semántica

1. La semántica es el conjunto de reglas que describe el significado de los símbolos, palabras y frases de un lenguaje, ya sea natural o informático.
2.
 - a. Compilar un programa es traducir el código fuente escrito en un lenguaje de alto nivel a un lenguaje de bajo nivel (código máquina), para que la computadora pueda ejecutarlo.
 - b. Pasos para compilar un programa:
 1. Análisis léxico (tokenización): se leen los caracteres del código fuente y se agrupan en tokens (palabras clave, identificadores, operadores, etc.).
Ej.: `int x = 5;` → `int, x, =, 5, ;`
 2. Análisis sintáctico (verificación de estructura): verifica que los tokens estén bien estructurados según la gramática del lenguaje.
Ej.: `int x = 5;` → BIEN / `int = x 5;` → MAL
 3. Análisis semántico (verificación de significado): comprueba que el programa tenga sentido lógico.
Ej.: variables declaradas, tipos compatibles. → `int x = "hola";` // MAL
 4. Generación de código intermedio: se transforma el programa en una representación intermedia (más simple que el código fuente).
 5. Generación de código objeto: se traduce el código intermedio a código máquina.
 6. Enlazado: se unen distintos archivos y bibliotecas necesarias para formar el programa ejecutable final.
 - c. La semántica interviene en el paso de análisis semántico. Su importancia radica en verificar que el programa tenga coherencia y sentido lógico, más allá de estar bien escrito, controlando aspectos como tipos de datos, declaraciones de variables y uso correcto de las operaciones.
3. No es lo mismo compilar que interpretar. Ambos procesos traducen un programa pero lo hacen de manera diferente.

- Diferencias básicas:

Compilación	Interpretación
• Traduce todo el programa a código máquina antes de ejecutarlo. • Genera un archivo ejecutable. • Detecta errores antes de ejecutar. • Ejecución más rápida.	• Traduce y ejecuta el programa línea por línea. • No genera ejecutable. • Detecta errores durante la ejecución. • Ejecución más lenta.

- **Compilación**

- Ventajas:
 - Mejor velocidad de ejecución
 - Detección de errores antes de correr el programa
 - Código optimizado
- Desventajas:
 - Proceso de compilación puede ser lento
 - Menos flexible ante cambios (hay que recompilar)

- **Interpretación**

- Ventajas:
 - Más flexible (no necesita recompilar)
 - Más fácil de probar y depurar
 - Ejecución inmediata
- Desventajas:
 - Menor rendimiento
 - Errores aparecen en tiempo de ejecución
 - Depende del intérprete

4. Un *error sintáctico* ocurre cuando el código no respeta las reglas gramaticales del lenguaje, impidiendo que el compilador lo procese. Por ejemplo, la falta de un punto y coma. En cambio, un *error semántico* ocurre cuando el código está bien escrito pero carece de sentido lógico, como asignar un valor de tipo incorrecto a una variable.

5. Problemas:

a. Pascal

- Sintáctico:
 - Program P → falta un punto y coma.
 - a=a+1; → faltan 2 puntos “:=”
- Semántico:
 - var 5 → las variables no pueden comenzar con número.
 - for i:= 5 to 10 do → i no está declarada
 - a:=a+1; → a es char y estás sumando un número. Incompatibilidad de tipos.

b. Java

- Sintáctico:
 - arrayList → es con mayúscula, ArrayList
 - (BOOLEAN) → está mal escrito, debería ser Boolean
 - listado.get(...) = ... → asignación inválida

// me falta uno

- Semántico:
 - for(i = 1; i < 11 ; i –) → i no está declarada
 - return true; → retorna un true pero debería retornar un String
 - listado.get(listado.size()-1)= (BOOLEAN) numero>i; → intenta asignar un valor a un lugar de la lista pero no se asigna con get
- Lógico:
 - el for comienza en 1, hasta 11 pero se decrementa, nunca llegará a 11 es infinito

c. C

- Sintáctico:
 - Llama a la función final */ → es un comentario que le falta /* al principio
 - numero_cuadrado == numero * numero; → se utiliza == (comparación) en vez de = (asignación)

- Semántico:
 - la función final() no existe definición
 - encabezado; → no es variable ni función válida
 - cuadrado (numero) → sin prototipo

// faltan semánticos

d. Python

- Sintáctico:
 - print sin parentesis
 - mod no es un operador válido, debe ser con %
- Semántico:
 - while r = True: → debería ser == de comparación
 - el % se puede utilizar si el string tiene %d o %s (especificadores)
 - N = input(...) devuelve un string y luego se utiliza en operaciones (% , <=, etc). Debería ser N = int(input(...))

e. Ruby

- Sintáctico:
 - Puts debe ser puts
 - comillas mal usadas en Juan
 - + nom mal usado
- Semántico:
 - edad no inicializada
 - multiplicación de strings incorrecta
 - 'días * 24' como string
 - uso incorrecto en tipos de operaciones

6. Java

```
public void ordenar_arreglo(char[] arreglo, int cont) {
    int i;
    boolean ordenado;
    char aux;

    do {
        ordenado = true;
        for(i = 0; i < cont-1; i++) {
```

```

        if (arreglo[i] > arreglo[i + 1]) {
            aux = arreglo[i];
            arreglo[i] = arreglo[i + 1];
            arreglo[i + 1] = aux;
            ordenado = false;
        }
    }
} while (!ordenado);
}

```

El algoritmo es una variante del ordenamiento:

- Recorre el arreglo intercambiando elementos adyacentes si estan desordenados
 - Repite el proceso hasta que no haya mas intercambios (ordenado = true)
- Arreglos
 - Sintaxis: en Java se declara primero el tipo y luego el nombre: char[] arreglo;
 - Semántica: es una estructura indexada que almacena elementos del mismo tipo.
 - Los índices comienzan en 0, a diferencia de Pascal que suelen comenzar en 1
 - El acceso se realiza con arreglo[i]
 - Variables
 - Sintaxis: se declara primero el tipo y luego el nombre: int i;, boolean ordenado; char aux;
 - Semántica: Java es fuertemente tipado, por lo que cada variable debe tener un tipo definido antes de usarse
 - for
 - Sintaxis: for (inicializacion ; condicion ; incremento)


```
for (i = 0; i < cont ; i ++)
```
 - Semántica: ejecuta un bloque de codigo mientras se cumple la condición. A diferencia de Pascal:
 - Se debe especificar explícitamente la inicializacion, condicion y actualización
 - No esta implícito el incremento
 - do - while (equivalente a repeat - until):
 - Sintaxis: do {


```
    // codigo
} while (condición) ;
```
 - Semántica:
 - Ejecuta el bloque al menos una vez
 - La condición se evalua al final

- Equivale a repeat... until de Pascal, pero con la condición negada:
 - Pascal: until ordenado
 - Java: while (! ordenado)
 - if:
 - Sintaxis:


```
if (condición) {
    // acciones
}
```
 - Semántica: ejecuta el bloque solo si la condición es verdadera.
 - Comparacion de caracteres:
 - en Pascal se usa ord() para obtener el valor ASCII
 - En Java no hace falta porque los char se pueden comparar directamente
if (arreglo[i] > arreglo[i + 1])

7. *self* en Ruby

- Semántica
 - representa la referencia al objeto actual en el contexto donde se esta ejecutando el codigo (contexto dinámico)
 - su valor depende del contexto
- Que valor toma?
 - Dentro de un objeto → self es el objeto
 - Dentro de una clase → self es la clase
 - En el contexto global → self es el objeto principal (main)
- Uso en el lenguaje
 - Permite acceder a atributos y metodo del objeto actual
 - Se usa para evitar ambigüedades al llamar métodos o variables
 - Define métodos de clase → def self.metodo_clase
end

Ejemplo:

```
class Persona
  def mostrar
    puts self ←self es la instancia de Persona
  end
end
```

nil en Ruby

- Semántica
 - representa la ausencia de valor
 - es un objeto especial de la clase NilClass
- Que valor toma?
 - tiene un único valor posible: nil
 - es equivalente a “null” en otros lenguajes
- Uso en el lenguaje

- indica que una variable no tiene valor asignado
- es el valor por defecto de métodos que no retornan nada explícito
- se usa en evaluaciones booleanas: nil es falso en condiciones (junto con false)

Ejemplo:

```
x = nil
if x
  puts "Verdadero"
else
  puts "Falso"
end
```

← imprime "Falso" porque nil se evalúa como falso

8. *undefined* en JavaScript

- Semántica: representa una variable que ha sido declarada pero no inicializada o una expresión sin valor definido. "Existe pero todavía no tiene valor asignado". Este valor es asignado automáticamente.
- Aparece en:
 - variables declaradas sin asignación → let x;
 - acceso a propiedades inexistentes → obj.prop
 - funciones sin return

null en JavaScript

- Semántica: representa la ausencia intencional de valor. "No hay valor y eso es intencional".
- Se usa cuando
 - se quiere indicar explícitamente que no hay valor → let x = null;

Diferencias:

Concepto	<i>undefined</i>	<i>null</i>
Origen	Automático (lo asigna JS)	Explícito (lo asigna programador)
Significado	No inicializado	Ausencia intencional
Tipo	undefined	object

Ejemplo:

```
let a, b = null;
console.log(a); // undefined ← implícito
console.log(b); // null      ← intencional
```

9. Semántica general de *break*: rompe la ejecución de estructuras de control, como bucles o switch, y transfiere el control a la instrucción siguiente.

En C:

- Semántica: finaliza la ejecución de bucles (for, while, do-while) y estructuras switch
- Características: rompe el bloque mas interno.

`break;`

En PHP:

- Semántica: interrumpe bucles y estructuras switch
- Características: puede romper múltiples niveles

`break 2;` → sale de dos estructuras anidadas

Similar a C, pero mas flexible

En JavaScript:

- Semántica: termina la ejecución de bucles (for, while, do...while) y switch
- Características: puede usarse con labels (etiquetas)

`break etiqueta;` → permite salir de bloques específicos

Sin etiqueta rompe solo el bloque mas interno

En Ruby:

- Semántica: interrumpe la ejecución de bucles (while, until, for, iteradores como each)
- Características: puede devolver un valor al salir del bucle

`break valor`

Diferencias

- C: basico, solo rompe el nivel actual
- PHP: permite romper múltiples niveles
- JavaScript: permite etiquetas
- Ruby: puede devolver un valor al salir

10. La ligadura es la asociación entre un nombre y un atributo (valor, tipo, dirección de memoria, función, método) y es clave para definir la semántica de un programa, ya que la ligadura define el significado de los elementos del programa. Gracias a ella el lenguaje “sabe”:

- que valor tiene una variable

- que tipo posee
- a que función se esta llamando

Sin ligaduras el programa no tendría un comportamiento definido.

- Ligadura estática:
 - se realiza en tiempo de compilación
 - NO cambia durante la ejecución
 - mas eficiente porque no necesita resolverse en ejecución
 - mas segura porque los errores son detectados antes

Ejemplos:

1. `int x = 10;`
 - el tipo de x (int) queda definido en compilación
2. `int suma(int a, int b) { return a + b; }`
 - la función a ejecutar se conoce en compilación

- Ligadura dinámica:
 - se realiza en tiempo de ejecución
 - puede cambiar durante la ejecución del programa
 - mas flexible
 - menor eficiente que la estática

Ejemplo de Java - polimorfismo:

1. `Animal a = new Perro();`
`a.hacerSonido();`
 - El método ejecutado depende del objeto real (Perro) no de la referencia (Animal)

Ejemplo en JavaScript:

2. `let x;`
`x = 5;`
`x = "hola";`
 - el tipo de x se decide en tiempo de ejecución